

SPAN: Spatial Pyramid Attention Network for Image Manipulation Localization

Authors: X. Hu et al.
Presenter: Trishita Patra

April 30, 2026

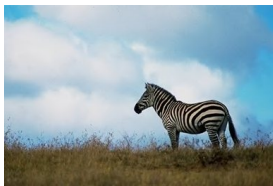
Introduction

- ▶ Image manipulation is the process of altering a digital image's content or appearance—such as adding, removing, or modifying regions—often to misrepresent reality or enhance visual quality.
- ▶ Image manipulation is widespread (fake news, fraud, research misuse).
- ▶ Detection prevents misinformation and deception.



Types of Manipulation

- ▶ **Splicing:** Combining content from different images.
- ▶ **Copy-Move:** Duplicating regions within the same image.
- ▶ **Removal:** Removing objects and filling background.



Examples



Motivation

Goal: Build a single model that can detect *multiple types of image manipulations simultaneously*.

- ▶ Existing methods are often specialized (splicing, copy-move, etc.).

Motivation

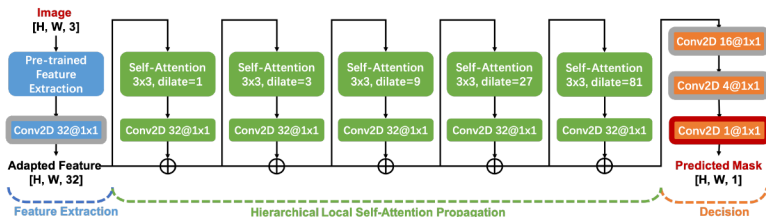
- ▶ Manipulated regions differ in **color, intensity, and noise distribution** from authentic regions.
- ▶ Assumption is that these subtle inconsistencies are **not visible to humans** but can be learned by models.

Key Idea (SPAN)

- ▶ Models both:
 - ▶ **Vertical (multi-scale)** relationships
 - ▶ **Spatial (patch-to-patch)** relationships
- ▶ Uses a pyramid of **local self-attention blocks**
- ▶ Detects inconsistencies **within a single image** (no original needed)

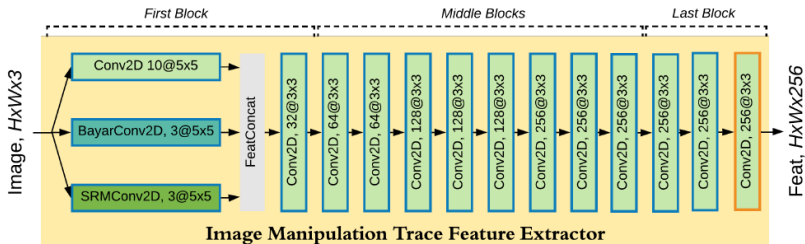
SPAN Overview

- ▶ Feature Extractor
- ▶ Pyramid Spatial Attention
- ▶ Decision Module



Feature Extractor

- ▶ Wide VGG backbone
- ▶ Bayer + SRM filters
- ▶ Extracts noise and visual features



Feature Extractor (from ManTra-Net)

Key Idea: Use a pretrained feature extractor trained on **385 manipulation types** (classification supervision).

- ▶ Trained on synthetic dataset (Dresden-based) to recognize diverse manipulation traces
- ▶ We **reuse and freeze** these features in SPAN (no retraining)
- ▶ Provides strong **forensic representation** before attention

Main Components:

- ▶ **Bayer Conv Layer:** Detects demosaicing inconsistencies across regions
- ▶ **SRM Filters:** Fixed high-pass filters that suppress image content and expose noise residuals (not learned)
- ▶ **Wide VGG Backbone:** Deep CNN using stacked 3×3 convs (efficient vs 5×5 , 7×7)

Output: Feature map of size $H \times W \times 256$

Feature Adaptation for SPAN

Problem: Attention cost scales as $O(D^2)$ $\rightarrow$ too expensive for $D = 256$

Solution: 1×1 Convolution (Channel Projection)

$$H \times W \times 256 \rightarrow H \times W \times 32$$

- ▶ 1×1 conv mixes **channels only** (no spatial change)
- ▶ Acts as a **learned dimensionality reduction**
- ▶ Reduces attention cost:

$$256^2 \rightarrow 32^2 \quad (64 \times \text{cheaper})$$

Why needed:

- ▶ Keeps most discriminative features
- ▶ Makes self-attention computationally feasible
- ▶ Preserves full spatial resolution for pixel-level prediction

Pyramid Spatial Attention: Overview

Goal: Capture both local and large-scale inconsistencies efficiently

Key Idea

- ▶ Each pixel attends to a small neighbourhood (local self-attention)
- ▶ Multiple such blocks are stacked to expand coverage
- ▶ Enables comparison of distant regions without global attention

Core Components

- ▶ Local Self-Attention (LSA) with Residual Connections
- ▶ Positional Projection
- ▶ Pyramid Propagation (multi-layer stacking)

Local Self-Attention

Key Idea

- ▶ Each pixel attends to a small neighbourhood (e.g., 3×3)
- ▶ Learns relationships between pixels (not just summation like CNNs)

Dilation & Coverage

- ▶ Dilation t controls spacing between neighbours
- ▶ $t = 1 \rightarrow$ immediate neighbours
- ▶ $t = 3 \rightarrow$ neighbours spaced farther apart
- ▶ Stacking layers increases coverage exponentially

$$\text{Coverage} = (2N + 1)^{2h}$$

- ▶ Small local window + multiple layers $\rightarrow$ large effective region

Local Self-Attention: Neighbourhood Example

3×3 Attention Window inside a 9×9 Grid

Dilation $t = 1$

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	*	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Dilation $t = 3$

0	0	0	0	0	0	0	0	0
0	1	0	0	1	0	0	1	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	1	0	0	*	0	0	1	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	1	0	0	1	0	0	1	0
0	0	0	0	0	0	0	0	0

- ▶ * = center pixel
- ▶ 1 = attended neighbours
- ▶ Dilation spreads the same 3×3 pattern over a larger region

Residual Connection in SPAN

Formulation

$$X' = \text{LSA}(X) + X$$

Why is it needed?

- ▶ Multiple attention layers can distort or overwrite original features
- ▶ Fine details (especially boundaries of manipulated regions) may get lost

What does residual connection do?

- ▶ Preserves original pixel-level information
- ▶ Allows each layer to only **refine** features instead of replacing them
- ▶ Maintains stability during deep stacking

Local Self-Attention Block

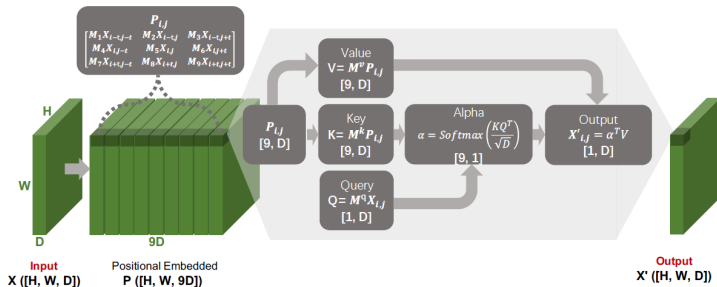


Fig. 3. Self-Attention Block: We perform self-attention mechanism over each target pixel $X_{i,j}$ and its local neighborhood $P_{i,j}$. With learnable projections M^q, M^k and M^v , we prepare Keys and Values from $P_{i,j}$ and Query from $X_{i,j}$ for the attention mechanism.

LSA Setup

Feature map:

$$X \in \mathbb{R}^{H \times W \times D}, \quad X_{i,j} \in \mathbb{R}^D$$

Neighbourhood (dilation t):

$$\begin{bmatrix} X_{i-t,j-t} & X_{i-t,j} & X_{i-t,j+t} \\ X_{i,j-t} & X_{i,j} & X_{i,j+t} \\ X_{i+t,j-t} & X_{i+t,j} & X_{i+t,j+t} \end{bmatrix}$$

Flatten:

$$P_{i,j} = [Y_1, \dots, Y_9] \in \mathbb{R}^{D \times 9}$$

Local Self-Attention: Q, K, V

$$Q = M^q X_{i,j}, \quad K = M^k P_{i,j}, \quad V = M^v P_{i,j}$$

- ▶ Query: what the pixel is looking for
- ▶ Key: what each neighbour offers
- ▶ Value: information to aggregate

Dimensions:

- ▶ $Q \in \mathbb{R}^{1 \times D}$
- ▶ $K, V \in \mathbb{R}^{9 \times D}$

Local Self-Attention Equation

$$\text{LSA}(X_{i,j}) = \frac{1}{C} \sum_{l=1}^9 \exp\left(\frac{\langle M^k Y_l, M^q X_{i,j} \rangle}{\sqrt{D}}\right) M^v Y_l$$

$$\text{where } C = \sum_{l=1}^9 \exp\left(\frac{\langle M^k Y_l, M^q X_{i,j} \rangle}{\sqrt{D}}\right)$$

- ▶ Dot product $\rightarrow$ similarity between pixel and neighbour
- ▶ Softmax $\rightarrow$ attention weights
- ▶ Output $\rightarrow$ weighted combination of neighbours

Key Insight

- ▶ Convolution: fixed weighted sum
- ▶ Attention: learns relationships between pixels
- ▶ Such relational information cannot be easily extracted by CNNs with similar parameters

Positional Projection

Problem

- ▶ Local Self-Attention is **position-agnostic**
- ▶ Same features at different locations $\rightarrow$ same attention weight
- ▶ But spatial position (left, right, diagonal) matters in images

Transformer Approach

$$\tilde{X} = X + PE(\text{pos})$$

- ▶ Adds positional embedding vector
- ▶ Works in NLP (semantic embedding space)
- ▶ Not suitable here (noise/texture features do not support addition)

SPAN Approach

- ▶ Use **learnable projection matrices** $M_l \in \mathbb{R}^{D \times D}$
- ▶ One matrix per relative position (9 for 3×3)
- ▶ Position encoded by **multiplication**, not addition

Positional Projection: Modified Attention

$$\text{LSA}(X_{i,j}) = \frac{1}{C} \sum_{l=1}^{(2N+1)^2} \exp\left(\frac{\langle M^k \mathbf{M}_l Y_l, M^q X_{i,j} \rangle}{\sqrt{D}}\right) M^v Y_l$$

$$C = \sum_{l=1}^{(2N+1)^2} \exp\left(\frac{\langle M^k \mathbf{M}_l Y_l, M^q X_{i,j} \rangle}{\sqrt{D}}\right)$$

Key Points

- ▶ M_l modifies **Key** only $\rightarrow$ affects attention weights
- ▶ Value $M^v Y_l$ remains unchanged
- ▶ All M_l are **learnable**

Boundary Handling

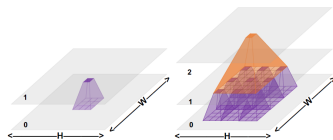
- ▶ Edge pixels $\rightarrow$ 6 neighbours
- ▶ Corner pixels $\rightarrow$ 4 neighbours
- ▶ SoftMax normalization adapts automatically (no padding needed)

Pyramid Propagation

Problem: Single local block sees only small region

Solution: Stack multiple layers with dilation, for $N = 1$,
 $2N + 1 = 3$ (3×3 nbd)

- ▶ Layer 1 ($t = 1$): 3×3
- ▶ Layer 2 ($t = 3$): 9×9
- ▶ Layer 3 ($t = 9$): 27×27
- ▶ Layer 4 ($t = 27$): 81×81
- ▶ Layer 5 ($t = 81$): 243×243



$$\text{Coverage} = (2N + 1)^{2h}$$

Why Pyramid Propagation Works

Benefit 1

- ▶ Small neighbourhood (3×3) $\rightarrow$ computationally efficient

Benefit 2

- ▶ Higher layers represent larger regions
- ▶ Each pixel encodes information from a patch
- ▶ Comparing pixels = comparing patches

Result

- ▶ Efficient multi-scale modeling
- ▶ Detects inconsistencies across distant regions

Block Size Analysis (Why 3×3 ?)

Coverage Requirement

$$M^h \geq S \quad \Rightarrow \quad h = \mathcal{O}(\log_M S)$$

Total Cost

$$\mathcal{O}(S^2 M^2 \log_M S) = \mathcal{O}\left(\frac{S^2 M^2 \ln S}{\ln M}\right)$$

Minimize

$$f(M) = \frac{M^2}{\ln M}$$

$$\frac{d}{dM} f(M) = 0 \Rightarrow \ln M = \frac{1}{2} \Rightarrow M = \sqrt{e} \approx 1.65$$

Conclusion

- ▶ M must be odd integer $\rightarrow$ choose $M = 3$
- ▶ 3×3 is most efficient
- ▶ 3×3 vs 5×5 : $\sim 1.9\times$ cheaper
- ▶ Block size is a **fixed hyperparameter** (not learnable).

Decision Module

Architecture

- ▶ Conv2D (16 @ 1×1)
- ▶ Conv2D (4 @ 1×1)
- ▶ Conv2D (1 @ 1×1) + Sigmoid

Output

$$X^{\text{pred}} \in [0, 1]^{H \times W}$$

(pixel-wise tampering probability map)

Why 1×1 Convolutions?

- ▶ Operate per-pixel (no spatial mixing)
- ▶ Preserve sharp boundaries of manipulated regions
- ▶ Act as channel combiners (feature selection)

Channel Compression

$$32 \rightarrow 16 \rightarrow 4 \rightarrow 1$$

- ▶ Progressive distillation of attention features

Training Framework & Loss

Ground Truth

$$B \in \{0, 1\}^{H \times W}$$

Binary Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{HW} \sum_{i,j} \left[B_{i,j} \log X_{i,j}^{\text{pred}} + (1 - B_{i,j}) \log(1 - X_{i,j}^{\text{pred}}) \right]$$

Why BCE?

- ▶ Pixel-wise binary classification (tampered vs authentic)
- ▶ Strong penalty for confident wrong predictions
- ▶ Probabilistically grounded (Bernoulli likelihood)

Why Normalization ($\frac{1}{HW}$)?

- ▶ Makes loss independent of image size
- ▶ Stabilizes gradient magnitude
- ▶ Ensures equal contribution from all pixels

Datasets

Three benchmark datasets evaluated (no fine-tuning):

▶ **Columbia Dataset**

- ▶ Type: Splicing detection
- ▶ 180 images
- ▶ Pixel-level ground truth masks

▶ **Coverage Dataset**

- ▶ Type: Copy-move forgery
- ▶ 100 images
- ▶ Pixel-level grayscale masks

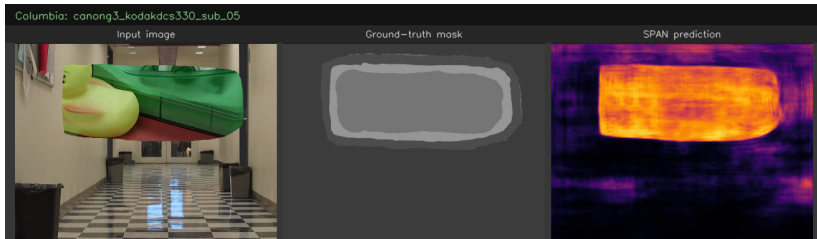
▶ **CASIA v1 Dataset**

- ▶ Type: Splicing + Copy-move
- ▶ 921 forged + 800 authentic
- ▶ **No pixel masks → image-level evaluation**

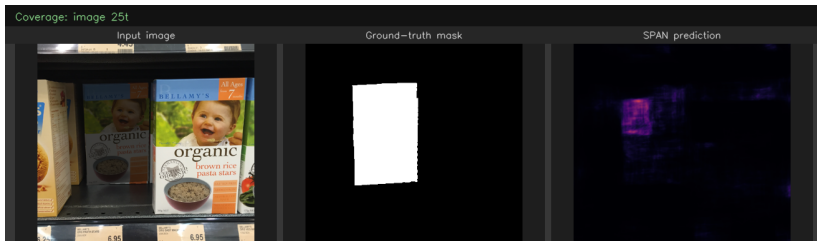
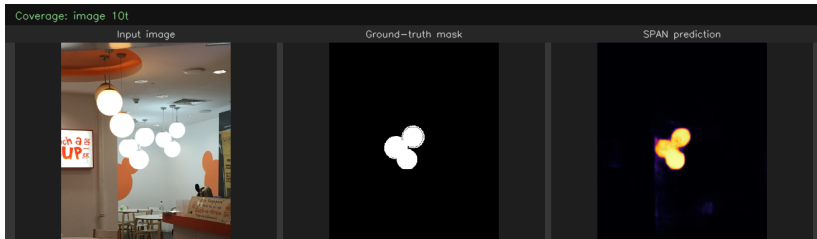
Evaluation difference:

- ▶ Columbia, Coverage → Pixel-level
- ▶ CASIA → Image-level (max pixel score)

Columbia Dataset — Sample Results



Coverage Dataset — Sample Results



Columbia Results (Pixel-Level AUC)

Condition	SPAN	ManTraNet	
No manipulation	92.86	72.09	+20.77
Resize 0.78×	84.30	62.30	+22.0
Resize 0.25×	65.12	61.03	+4.09
Blur (k=3)	66.95	62.57	+4.38
Blur (k=15)	66.07	63.77	+2.30
Noise =3	73.21	59.62	+13.59
Noise =15	67.84	51.94	+15.90
JPEG q=100	91.91	66.35	+25.56
JPEG q=50	71.03	57.70	+13.33

Observations:

- ▶ SPAN consistently outperforms across all perturbations
- ▶ Strongest when noise fingerprint is preserved (no compression / JPEG 100)

Coverage Results (Pixel-Level)

Condition	SPAN AUC	MT AUC	SPAN F1	MT F1
No manipulation	91.37	71.61	44.95	24.31
Resize 0.78×	74.05	65.16	21.85	21.71
Resize 0.25×	63.13	62.61	19.57	20.60
Blur (k=3)	65.06	64.45	19.74	22.38
Blur (k=15)	63.36	61.67	19.57	22.19
Noise =3	69.03	60.19	19.99	20.10
Noise =15	60.26	53.59	20.52	19.41
JPEG q=100	87.63	66.14	35.35	21.74
JPEG q=50	65.99	59.49	19.57	19.99

Observations:

- ▶ SPAN generalises well to copy-move
- ▶ F1 lower → boundaries harder than splicing

CASIA v1 Results (Image-Level)

Condition	SPAN AUC	MT AUC	SPAN F1	MT F1
No manipulation	73.11	47.96	63.14	69.72
Resize 0.78×	59.99	45.26	35.15	69.72
Resize 0.25×	48.67	45.79	36.86	69.72
Blur (k=3)	53.79	46.62	10.50	69.72
Blur (k=15)	49.18	47.13	6.81	69.72
Noise =3	48.72	49.44	67.21	69.72
Noise =15	52.00	57.35	69.65	69.72
JPEG q=100	72.54	48.03	59.54	69.72
JPEG q=50	60.46	41.76	44.27	69.67

Observations:

- ▶ SPAN AUC reasonable; lower due to image-level evaluation
- ▶ ManTraNet nearly constant F1 → poor discrimination

Overall Analysis & Implementation Notes

Robustness Trends

- ▶ Most robust: **JPEG (q=100)** → minimal signal loss
- ▶ Moderate drop: Gaussian noise, mild resize
- ▶ Worst case: **blur + aggressive resize** → destroys noise fingerprint

Why CASIA Performance is Lower

- ▶ No pixel-level supervision → weaker evaluation signal
- ▶ JPEG compression reduces forensic noise patterns

Implementation Notes

- ▶ Original models in TensorFlow (.h5) → version incompatibility
- ▶ Re-implemented in PyTorch using Copilot assistance
- ▶ Weights loaded via custom conversion (h5py)
- ▶ Paper used **FP32**; we used **FP16** → slight performance variation

Conclusion

What SPAN Demonstrates

- ▶ Combines CNN features + local self-attention
- ▶ Captures **spatial inconsistencies** across multiple scales

Key Findings

- ▶ Outperforms ManTraNet by up to **+20 AUC**
- ▶ Generalises without fine-tuning
- ▶ Pyramid attention improves large-region detection

Limitations

- ▶ Sensitive to heavy blur / resizing
- ▶ Depends on preservation of noise fingerprints

Thank you!